

01 / Gradient descent and coordinates

Coordinates and optimization

The same optimization problem can be written using different variables. Changing coordinates does not change which physical point has the lowest objective value, but it can change the route that gradient descent takes to reach it. This tutorial starts with a quadratic function, explains how the difference arises and how to correct it, and then considers how coordinates and scales can be chosen to improve updates.

Adam and SOAP continue this discussion: how should an update account for unequal gradient scales, and in which directions should those scales be measured?

A gradient describes local rates of change

Consider

$$f(x, y) = \frac{1}{2}[(x - 2)^2 + (y - 1)^2].$$

Both squared terms are nonnegative and vanish together at $(2, 1)$. The minimum is therefore 0, attained at $(2, 1)$. The gradient is

$$\nabla f(x, y) = \begin{pmatrix} \partial f / \partial x \\ \partial f / \partial y \end{pmatrix} = \begin{pmatrix} x - 2 \\ y - 1 \end{pmatrix}.$$

The partial derivative $\partial f / \partial x$ describes how the function changes with x while y is fixed. The gradient collects the partial derivatives into a column vector.

For a small displacement $\delta = (\delta x, \delta y)^\top$, the change in the function is approximately

$$f(x + \delta x, y + \delta y) - f(x, y) \approx (x - 2)\delta x + (y - 1)\delta y = \nabla f^\top \delta.$$

Choosing $\delta = -\eta \nabla f$ makes the right-hand side $-\eta \|\nabla f\|^2$. For $\eta > 0$ and a nonzero gradient, this first-order approximation predicts a decrease. The actual step must still be suitable: the approximation can become inaccurate over a large displacement.

The learning rate sets the scale of the step

Gradient descent starts from an initial point and repeats

$$x_{t+1} = x_t - \eta(x_t - 2), \quad y_{t+1} = y_t - \eta(y_t - 1).$$

The index t counts iterations, and η is the learning rate. At $(3, 2)$ the gradient is $(1, 1)^\top$. With $\eta = 0.2$, the first step reaches

$$(x_1, y_1) = (3 - 0.2, 2 - 0.2) = (2.8, 1.8).$$

For this function, the negative gradient points directly toward the minimum. With $0 < \eta < 1$, each step covers part of the remaining distance: both coordinate errors are multiplied by $1 - \eta$.

Two representations of one point

Now let $x = cu$, $y = v$, where $c > 1$. An increase of 1 in u corresponds to an increase of c in x . For example, when $c = 2$, the coordinates $(x, y) = (3, 2)$ and $(u, v) = (1.5, 2)$ describe the same point. Throughout the following comparisons, initial points correspond in this way, so a change of starting location does not get confused with a change of coordinates.

02 / Updates after a coordinate change

One function, different trajectories

Under $x = cu$, $y = v$, the objective becomes

$$g(u, v) = f(cu, v) = \frac{1}{2}[(cu - 2)^2 + (v - 1)^2].$$

Corresponding points satisfy $g(u, v) = f(x, y)$. The new-coordinate minimizer is $(2/c, 1)$, which maps back to $(2, 1)$. Rewriting the variables preserves objective values and the physical location of the minimum.

The chain rule changes the gradient

Differentiating the first term with respect to u introduces the derivative c of $c u - 2$:

$$\frac{\partial g}{\partial u} = \frac{1}{2} \cdot 2(cu - 2) \cdot c = c(cu - 2), \quad \frac{\partial g}{\partial v} = v - 1.$$

Using learning rate α in the new coordinates gives

$$u_{t+1} = u_t - \alpha c(cu_t - 2), \quad v_{t+1} = v_t - \alpha(v_t - 1).$$

To compare this with the original algorithm, map its updated points back to (x, y) . Write $\tilde{x}_t = cu_t$ and $\tilde{y}_t = v_t$. Then

$$\tilde{x}_{t+1} = \tilde{x}_t - \alpha c^2(\tilde{x}_t - 2), \quad \tilde{y}_{t+1} = \tilde{y}_t - \alpha(\tilde{y}_t - 1).$$

The horizontal coefficient is αc^2 , while the vertical coefficient is α . One factor of c comes from the chain rule. The other converts the displacement in u into a displacement in x .

One learning rate cannot compensate for both scales

Starting from $(3, 2)$ and its corresponding point $(3/c, 2)$, the first steps are

$$(x_1, y_1) = (3 - \eta, 2 - \eta), \quad (\tilde{x}_1, \tilde{y}_1) = (3 - \alpha c^2, 2 - \alpha).$$

Agreement requires both $\alpha c^2 = \eta$ and $\alpha = \eta$. For $c > 1$ and $\eta > 0$, these conditions are incompatible.

With $c = 2$ and $\eta = 0.2$, the difference is visible numerically:

Computation	Learning rate	First step in original coordinates
Original-coordinate descent	$\eta = 0.2$	$(2.8, 1.8)$
New-coordinate descent	$\alpha = 0.2$	$(2.2, 1.8)$
New-coordinate descent	$\alpha = 0.05$	$(2.8, 1.95)$

Reducing α corrects the horizontal update but also changes the vertical one. The mismatch concerns the update rule: corresponding objective values already agree, while ordinary gradient descent updates do not automatically correspond.

03 / Adjust each direction

Two scale factors and a correction matrix

In the original example, separate learning rates for u and v solve the mismatch:

$$u_{t+1} = u_t - \frac{\eta}{c^2} \frac{\partial g}{\partial u}, \quad v_{t+1} = v_t - \eta \frac{\partial g}{\partial v}.$$

The factor c^{-2} cancels the two factors of c from the chain rule and displacement conversion. The vertical coefficient remains η . Both recurrences now agree with the original algorithm.

Include a vertical scale factor

More generally, let

$$x = cu, \quad y = dv, \quad c, d > 0, \quad g(u, v) = f(cu, dv).$$

This calculation applies to any differentiable f . Abbreviate $f_x = \partial f / \partial x$ and $f_y = \partial f / \partial y$, evaluated at the corresponding point (cu, dv) . The chain rule gives

$$g_u = cf_x, \quad g_v = df_y.$$

If the two new coordinates share learning rate α , the mapped updates are

$$x^+ = x - \alpha c^2 f_x, \quad y^+ = y - \alpha d^2 f_y.$$

A superscript $+$ means the updated point. Restoring the original coefficient η for all gradient directions requires

$$\alpha c^2 = \eta, \quad \alpha d^2 = \eta.$$

Since $c, d > 0$, one common rate suffices only when $c = d$. Uniform scaling can be compensated uniformly; unequal scales need separate treatment. A zero gradient component imposes no constraint at that particular step. Here we require agreement of the entire update rule.

A matrix collects the two coefficients

Choose

$$\alpha_u = \frac{\eta}{c^2}, \quad \alpha_v = \frac{\eta}{d^2},$$

and write

$$z = \begin{pmatrix} u \\ v \end{pmatrix}, \quad \theta = \begin{pmatrix} x \\ y \end{pmatrix}, \quad A = \begin{pmatrix} c & 0 \\ 0 & d \end{pmatrix}.$$

The coordinate relation is $\theta = Az$, and the corrected update is

$$z^+ = z - \eta P \nabla g(z), \quad P = \begin{pmatrix} c^{-2} & 0 \\ 0 & d^{-2} \end{pmatrix}.$$

The matrix A changes coordinates; P acts on the gradient. Since P is diagonal, it only scales the components separately. When the coordinate change mixes variables, the correction may also need to combine different gradient components.

04 / General linear coordinates

Why the gradient uses a transpose

Collect the variables into column vectors $\theta, z \in \mathbb{R}^n$ and let

$$\theta = Az, \quad g(z) = f(Az),$$

where A is a fixed, invertible $n \times n$ matrix. Invertibility means that every θ has a unique representation $z = A^{-1}\theta$. The matrix can include scaling, rotation, and mixing between variables.

From the componentwise chain rule to a matrix formula

The i th component of $\theta = Az$ is

$$\theta_i = \sum_{j=1}^n A_{ij} z_j.$$

Changing one z_j can change several θ_i . Adding their contributions to the change in f gives

$$\frac{\partial g}{\partial z_j} = \sum_{i=1}^n \frac{\partial f}{\partial \theta_i} \frac{\partial \theta_i}{\partial z_j} = \sum_{i=1}^n A_{ij} \frac{\partial f}{\partial \theta_i}.$$

This is exactly the j th component of $A^\top \nabla_\theta f$. Therefore,

$$\nabla_z g(z) = A^\top \nabla_\theta f(Az).$$

The transpose $A^\top$ swaps the rows and columns of A . It appears because a derivative with respect to one new coordinate collects that coordinate's effects on all the original variables.

Gradients and displacements transform differently

For a displacement, the fixed linear relation directly gives

$$\delta\theta = A\delta z.$$

For a gradient, the chain rule gives $\nabla_z g = A^\top \nabla_\theta f$. These two relations enter at different stages of an update.

Ordinary gradient descent in z produces

$$\delta z = -\alpha \nabla_z g = -\alpha A^\top \nabla_\theta f.$$

Mapping that displacement back gives

$$\delta\theta = -\alpha AA^\top \nabla_\theta f.$$

In general, $AA^\top$ is not a scalar multiple of the identity matrix I , so one learning rate cannot remove its whole effect. For $A = \text{diag}(c, d)$, this recovers the factor $\text{diag}(c^2, d^2)$ from the previous page.

If A is orthogonal, then $AA^\top = I$. A pure rotation or reflection preserves Euclidean length, and ordinary gradient descent with the same learning rate gives corresponding updates. The issue therefore concerns how the distance rule is handled under a coordinate change, rather than arising from every possible change of coordinates.

05 / Recover the original update

The correction for a general transformation

In the original coordinates, the desired update is

$$\theta^+ = \theta - \eta \nabla f(\theta).$$

In the new coordinates, allow a matrix P to correct the gradient:

$$z^+ = z - \eta P \nabla g(z).$$

Using $\theta = Az$ and $\nabla g = A^\top \nabla f$, mapping back gives

$$Az^+ = Az - \eta APA^\top \nabla f(Az).$$

Agreement with the original rule for every gradient therefore requires

$$APA^\top = I.$$

Multiply on the left by A^{-1} and on the right by $(A^\top)^{-1}$:

$$P = A^{-1}(A^\top)^{-1} = (A^\top A)^{-1}.$$

The last equality uses $(BC)^{-1} = C^{-1}B^{-1}$. Matrix multiplication order matters: this result cannot generally be abbreviated as A^{-2} .

From one matching step to an entire matching trajectory

Starting from $\theta_0 = Az_0$, the corrected recurrence ensures $\theta_1 = Az_1$. Both methods are then at corresponding points again, so their next steps also correspond. Repeating the argument gives

$$\theta_t = Az_t$$

for every t . The comparison is made after expressing both methods in the same coordinates; the numerical vector z_t can of course differ from θ_t . The result holds for any differentiable objective, any fixed invertible linear transformation, and the same learning-rate sequence in both runs.

A correction may mix gradient components

Take $x = u + v$, $y = v$. Changing v now changes both x and y :

$$A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad A^\top A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad P = \begin{pmatrix} 2 & -1 \\ -1 & 1 \end{pmatrix}.$$

If $\nabla f = (1, 2)^\top$ at a corresponding point, then

$$\nabla g = A^\top \nabla f = \begin{pmatrix} 1 \\ 3 \end{pmatrix}, \quad P \nabla g = \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \quad AP \nabla g = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

The off-diagonal entries of P allow an update component to depend on several gradient components. Although the corrected vector is $(-1, 2)^\top$ in the new coordinates, mapping it back recovers $(1, 2)^\top$. Both algorithms subtract the same final displacement.

A matrix correction can therefore do more than assign one learning rate to each coordinate. This is a preconditioned update: a linear transformation of the gradient determines the parameter displacement.

06 / Metrics and step length

The matrix also describes distance

The correction matrix was obtained by matching updates algebraically. It has a geometric meaning too: when coordinate units change, the rule for measuring a step's length should change with them.

Under $x = cu$, $y = dv$, displacements satisfy $\delta x = c\delta u$ and $\delta y = d\delta v$. The squared length of the same displacement is therefore

$$(\delta x)^2 + (\delta y)^2 = c^2(\delta u)^2 + d^2(\delta v)^2.$$

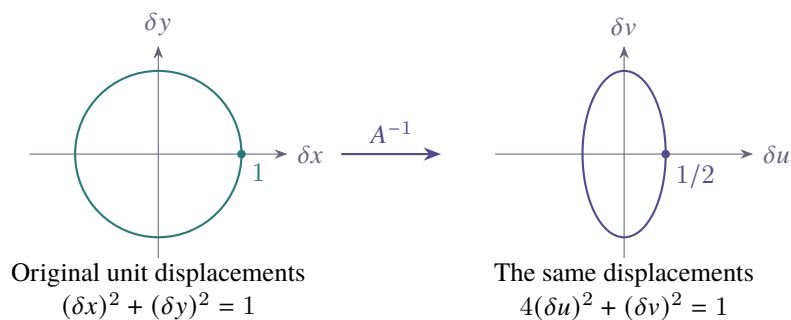
For a general transformation $\theta = Az$,

$$\|\delta\theta\|^2 = \|A\delta z\|^2 = \delta z^\top \underbrace{A^\top A}_M \delta z.$$

Here $\|\cdot\|$ is ordinary Euclidean length. The metric matrix M specifies how long a displacement in the new coordinates should count as.

Two shapes for the same unit displacements

With $c = 2$, $d = 1$, all displacements of original-coordinate length 1 satisfy $4(\delta u)^2 + (\delta v)^2 = 1$ in the new coordinates.



Moving $1/2$ along u and moving 1 along v both have original-coordinate length 1. The ellipse and circle represent the same set of displacements.

If we instead use $(\delta u)^2 + (\delta v)^2$ as the length rule, one unit along u and one along v count as equally long. Their lengths after mapping back are 2 and 1. This adopts a different rule for distance.

The metric and the preconditioner

For invertible A , we have $M^\top = M$, and for every nonzero δz ,

$$\delta z^\top M \delta z = \|A\delta z\|^2 > 0.$$

Such a matrix is symmetric positive definite. Positive definiteness gives positive length to every nonzero displacement and ensures that M is invertible.

The correction found by matching trajectories is exactly

$$P = M^{-1}.$$

The metric M measures displacements, while the preconditioner P corrects gradients. A local descent problem explains why these two matrices are inverses.

The picture here shows equal-length displacements. Later, ellipses will represent objective contours. These are different meanings of an elliptical picture.

07 / The metric gradient

Define descent using a distance rule

The meaning of steepest descent depends on how length is measured. The ordinary gradient uses Euclidean length. With the rule $\delta^\top M \delta$, candidate displacements must be compared using that metric.

At the current point z , a first-order approximation gives

$$g(z + \delta) - g(z) \approx \nabla g(z)^\top \delta.$$

Minimizing the right-hand side alone would allow arbitrarily large displacements. Add a penalty on squared step length to obtain a finite local problem:

$$\delta_* = \arg \min_{\delta} \left\{ \nabla g(z)^\top \delta + \frac{1}{2\eta} \delta^\top M \delta \right\}, \quad \eta > 0.$$

The first term favors a decrease in the objective; the second makes large displacements costly. A smaller η penalizes length more heavily and produces a smaller step.

Solve the local problem

Since M is symmetric, the derivative of the quadratic term is $\eta^{-1} M \delta$. Setting the full derivative to zero gives

$$\nabla g(z) + \eta^{-1} M \delta_* = 0,$$

and hence

$$\delta_* = -\eta M^{-1} \nabla g(z).$$

Positive definiteness makes this local minimizer unique. The original objective was approximated to first order, so actual descent still requires a suitable step size.

The vector $M^{-1} \nabla g$ is the gradient under metric M . In the inner product $\langle a, b \rangle_M = a^\top M b$, it satisfies

$$\langle M^{-1} \nabla g, \delta \rangle_M = \nabla g^\top \delta.$$

With the new distance rule, this vector still reproduces the first-order change along any displacement. Setting $M = I$ recovers the ordinary gradient. Setting $M = A^\top A$ recovers the coordinate correction.

The connection to natural gradient

Defining a gradient after choosing a metric is the construction used in Riemannian gradient methods. Natural gradient uses a metric determined by a statistical model, measuring parameter changes through changes in that model. It shares the geometric idea that a metric determines the gradient. An arbitrary preconditioner is not automatically a statistical natural gradient.^[1]

The coordinate correction here requires no statistical metric construction: M already follows from $\theta = Az$ and Euclidean length in the original coordinates. If those coordinates instead used a metric M_θ , the same length calculation would give $M_z = A^\top M_\theta A$.

The exact correspondence of discrete trajectories uses a fixed linear transformation. A nonlinear coordinate change generally uses its derivative to approximate displacement locally. Correctly transformed gradient directions alone need not give exactly corresponding points after a finite step.

08 / From agreement to efficiency

Choose a different update rule

Once two methods follow the same trajectory, they also have the same objective value at each iteration. They cannot require different numbers of steps to reach a specified loss. Their computational costs per step may differ, but matching iterates does not itself reduce the iteration count.

Seeking faster convergence means allowing a different trajectory. A coordinate change can help design such an update: select coordinates and adopt ordinary Euclidean gradient descent there.

Run ordinary descent in another coordinate system

Use B for this new choice, to distinguish it from the earlier transformation A :

$$\theta = Bw, \quad \ell(w) = f(Bw),$$

where B is fixed and invertible. Ordinary gradient descent in w gives

$$\nabla_w \ell = B^\top \nabla_\theta f, \quad \delta w = -\eta B^\top \nabla_\theta f.$$

Mapping back,

$$\delta \theta = B \delta w = -\eta \underbrace{BB^\top}_P \nabla_\theta f.$$

Thus, adopting Euclidean gradient descent in the new coordinates is equivalent to choosing preconditioner $P = BB^\top$ in the original coordinates.

Conversely, every symmetric positive definite matrix can be written as $BB^\top$. For example, $P = \text{diag}(0.01, 1)$ factors using $B = \text{diag}(0.1, 1)$. Its preconditioned update can be interpreted as ordinary gradient descent in the corresponding new coordinates.

Two uses of a coordinate transformation

Previously, we preserved the existing distance rule. After changing coordinates with A , we used $M_z = A^\top A$ and corrected the gradient by M_z^{-1} . Mapping back then recovered the original ordinary gradient descent update.

Now we permit a new distance rule. Using the identity as the metric in w produces $BB^\top$ after mapping back. This matrix generally differs from I , so the update usually changes the trajectory.

This distinction explains why one formula contains an inverse and the other does not:

$$\text{recover an update: } P_z = (A^\top A)^{-1}, \quad \text{choose an update: } P_\theta = BB^\top.$$

They act on gradients in different coordinate systems and serve different comparison goals.

Changing a picture and changing an algorithm

Stretching a contour plot horizontally can make an ellipse look circular, but it does not change the update already being computed. To affect optimization, we must actually use the chosen rule in the new coordinates, or apply its equivalent preconditioner in the original ones.

For a fixed matrix, this relationship is exact. A preconditioner that changes during optimization need not correspond to one fixed global coordinate transformation. Adaptive optimizers continually adjust scales or directions as their iterations proceed.

09 / Contours and descent directions

How ellipses and circles describe an objective

A contour consists of points with the same function value. For the original quadratic and any $k > 0$,

$$f(x, y) = k \iff (x - 2)^2 + (y - 1)^2 = 2k.$$

This is a circle centered at $(2, 1)$. Smaller circles have smaller k and lower objective values. Under $x = cu$, $y = dv$, the same value is represented by

$$c^2(u - 2/c)^2 + d^2(v - 1/d)^2 = 2k.$$

Its semiaxis lengths are $\sqrt{2k}/c$ and $\sqrt{2k}/d$. When $c \neq d$, it is an ellipse. A larger coefficient gives a shorter semiaxis: a small movement in that direction can change the objective substantially.

The negative gradient usually misses the minimum

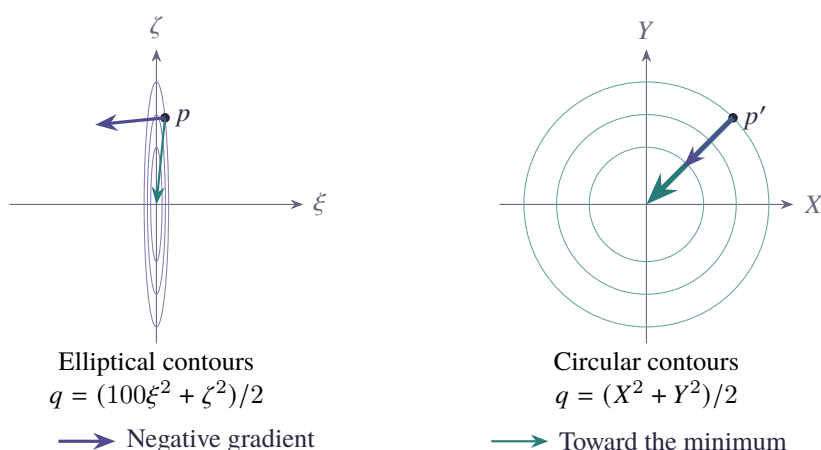
A small tangent displacement along a contour has zero first-order change in the objective. The gradient is therefore perpendicular to the contour's tangent. Its negative gives the fastest local decrease when directions are compared using Euclidean length.

This direction uses information near the current point and need not point directly toward the minimum. On a narrow ellipse it tends to favor the steeper direction. For a spherical quadratic, the negative gradient instead follows a radius toward the center.

Set $c = 10$, $d = 1$ and shift the minimizer to the origin:

$$\xi = u - 0.2, \quad \zeta = v - 1, \quad q(\xi, \zeta) = \frac{1}{2}(100\xi^2 + \zeta^2).$$

Represent the same point using $X = 10\xi$ and $Y = \zeta$. Then $q = \frac{1}{2}(X^2 + Y^2)$.



Each panel uses equal horizontal and vertical units. Arrows show directions, not the actual displacement for a specified learning rate.

On the left, the gradient is $(100\xi, \zeta)^\top$, while the vector to the center is $(-\xi, -\zeta)^\top$. The negative gradient is usually not parallel to that vector. On the right, the gradient is $(X, Y)^\top$, so its negative points directly to the center.

The directional difference explains only part of the difficulty. More directly, different directions of the ellipse need different step sizes, while ordinary gradient descent shares one learning rate across them.

10 / A shared learning rate

How slow directions arise

For

$$q(\xi, \zeta) = \frac{1}{2}(100\xi^2 + \zeta^2),$$

ordinary gradient descent gives

$$\xi_{t+1} = (1 - 100\alpha)\xi_t, \quad \zeta_{t+1} = (1 - \alpha)\zeta_t.$$

Each error component is multiplied by a fixed factor. For every initial error to approach zero, both factors must have magnitude below 1:

$$|1 - 100\alpha| < 1, \quad |1 - \alpha| < 1.$$

The first condition requires $0 < \alpha < 0.02$; the second requires $0 < \alpha < 2$. A shared learning rate must satisfy the stricter bound, $0 < \alpha < 0.02$.

The steeper direction restricts the rate for every direction

The original problem starts at $(\xi_0, \zeta_0) = (0.1, 1)$ in these shifted coordinates. With $\alpha = 0.01$, its first step is

$$(0.1, 1) \mapsto (0, 0.99).$$

The horizontal coordinate reaches its optimum immediately, while the vertical error decreases by only 1%. Subsequently, $\zeta_t = 0.99^t$:

Iteration t	0	1	10	100	230
Vertical error $ \zeta_t $	1	0.99	0.9044	0.3660	0.0991

The error first falls below 0.1 after 230 steps. This calculation shows how a small learning rate can stabilize one direction while leaving slow progress in another.

With $\alpha = 0.019$, the horizontal factor is -0.9 : the error alternates signs but shrinks. With $\alpha = 0.03$, it is -2 , so the horizontal error reverses and grows. The vertical coordinate is still stable at that rate, but the full iteration no longer converges.

The circular quadratic permits a suitable common scale

In coordinates $X = 10\xi$, $Y = \zeta$,

$$q = \frac{1}{2}(X^2 + Y^2), \quad (X^+, Y^+) = (1 - \eta)(X, Y).$$

Both directions have the same factor, and every $0 < \eta < 2$ gives convergence. Choosing $\eta = 1$ reaches the origin in one step. Mapping back to (ξ, ζ) gives

$$\Delta \begin{pmatrix} \xi \\ \zeta \end{pmatrix} = -\eta \begin{pmatrix} 0.01 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 100\xi \\ \zeta \end{pmatrix} = -\eta \begin{pmatrix} \xi \\ \zeta \end{pmatrix}.$$

Running descent after the rescaling gives both original directions suitable actual steps. This result is exact for the known quadratic. A general objective need not have constant curvature, and nearly circular contours alone do not guarantee a one-step solution.

A small gradient by itself does not imply that the update should be enlarged; it can also mean proximity to the minimum. Here the slow progress comes from unequal curvatures restricting a common learning rate.

11 / Axes and principal directions

Rotation separates mixed directions

The preceding ellipse was aligned with the axes, allowing the horizontal and vertical coordinates to be treated separately. In a general problem, the fastest and slowest directions may be tilted. Scaling the original coordinates separately may then fail to express the desired correction.

Consider

$$q(\theta) = \frac{1}{2}\theta^\top H\theta, \quad H = \begin{pmatrix} 50.5 & 49.5 \\ 49.5 & 50.5 \end{pmatrix}, \quad \theta = \begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix}.$$

Expanding gives

$$q(\theta) = \frac{1}{2}(50.5\theta_1^2 + 99\theta_1\theta_2 + 50.5\theta_2^2).$$

The cross term $\theta_1\theta_2$ shows that the coordinates do not separate the principal directions. The Hessian H is the matrix of second partial derivatives. The gradient of this quadratic is $\nabla q = H\theta$.

Eigenvectors identify directions that do not mix

Take the orthogonal unit vectors

$$q_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad q_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix}.$$

They satisfy $Hq_1 = 100q_1$ and $Hq_2 = q_2$. Multiplication by H preserves these directions and changes only their lengths, so they are eigenvectors. The factors 100 and 1 are their eigenvalues.

Collect them as columns:

$$Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad \Lambda = \text{diag}(100, 1).$$

Then $Q^\top Q = QQ^\top = I$ and $H = Q\Lambda Q^\top$. In the coordinates $r = Q^\top\theta$,

$$q = \frac{1}{2}r^\top \Lambda r = \frac{1}{2}(100r_1^2 + r_2^2).$$

The components r_1, r_2 measure position along the two principal directions. The cross term disappears, returning us to an axis-aligned ellipse.

Rotation preserves the curvature ratio

Moving the same distance along the first principal direction produces a larger quadratic objective increase than moving along the second. For a positive definite matrix, this difference is measured by the condition number:

$$\kappa(H) = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)} = 100.$$

The quantities $\lambda_{\max}$ and $\lambda_{\min}$ are the largest and smallest eigenvalues. Orthogonal rotation preserves them, and therefore preserves the condition number.

Rotation identifies directions that can be treated separately. Removing their unequal scales requires a further rescaling.

12 / Whitening and preconditioning

Rescale the principal directions

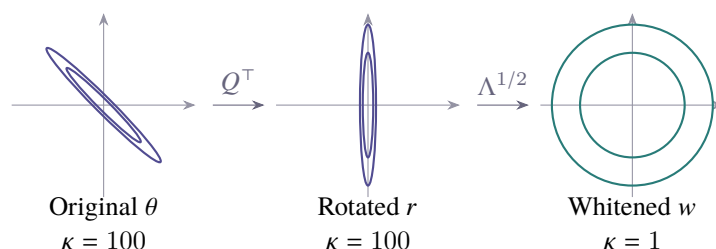
After rotation, the objective is

$$q = \frac{1}{2}(100r_1^2 + r_2^2).$$

Set $w_1 = 10r_1$, $w_2 = r_2$, or in matrix form $w = \Lambda^{1/2}r$. This gives

$$q = \frac{1}{2}(w_1^2 + w_2^2) = \frac{1}{2}w^\top w.$$

The matrix $\Lambda^{1/2} = \text{diag}(10, 1)$ takes the square root of each positive eigenvalue. Rotation followed by scaling transforms this positive definite quadratic to one with curvature 1 in every direction. This is whitening.



All three panels show corresponding objective values. Rotation aligns the principal axes; scaling then turns the ellipse into a circle.

The preconditioner corresponding to whitening

Since $r = Q^\top \theta$, the map from w back to the original parameters is

$$\theta = Q\Lambda^{-1/2}w.$$

Using the notation of Section 8, $B = Q\Lambda^{-1/2}$. Ordinary gradient descent in w therefore maps back to

$$P = BB^\top = Q\Lambda^{-1}Q^\top = H^{-1}.$$

For this example,

$$P = \begin{pmatrix} 0.505 & -0.495 \\ -0.495 & 0.505 \end{pmatrix}, \quad \delta\theta = -\eta H^{-1} \nabla q.$$

Since $\nabla q = H\theta$, the update is $\theta^+ = (1-\eta)\theta$. Choosing $\eta = 1$ reaches the minimum in one step. The off-diagonal entries show why correcting a tilted ellipse generally requires mixing original-coordinate components.

Better coordinates have computational costs

Here H is known, fixed, and positive definite, so exact whitening is available. For a general nonlinear objective, the Hessian changes with position and may have negative eigenvalues. A transformation useful at one point need not suit the entire optimization.

Even a suitable matrix costs computation to estimate, store, factor, and apply. Efficiency should be measured by time to a common loss or accuracy target, with attention to gradient evaluations and memory. Fewer iterations do not necessarily mean less elapsed time.

These costs motivate estimating update scales from information that is easier to obtain. Training already computes a gradient at each step. Some historical statistics can adjust the update continually without constructing the full Hessian. Adam is one example.

13 / Adam

Estimate scales from gradient history

Whitening above used known curvature. During neural-network training, full curvature information is usually difficult to obtain, while present and past gradients are already available. Adam keeps two historical averages for each coordinate and uses them to construct updates.[2]

Write the step- t gradient as $a_t = \nabla \mathcal{L}(\theta_{t-1})$. Here $\mathcal{L}$ is the loss being optimized; in minibatch training, a_t can be the gradient estimate from the current batch. Adam maintains

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) a_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(a_t \odot a_t).$$

Initialize $m_0 = v_0 = 0$, with $0 \leq \beta_1, \beta_2 < 1$. The product $\odot$ is elementwise, so $a_t \odot a_t$ contains the squared gradient components.

What the two averages record

The exponential moving average m_t is the first-moment estimate. For example, $\beta_1 = 0.9$ retains 90% of the old state and adds 10% of the current gradient. Persistent directions accumulate, while alternating signs partly cancel.

The second-moment estimate v_t averages squared gradients. Squaring removes the sign, recording recent coordinate scales. Its square root has the same units as the gradient and serves as a normalization denominator.

Zero initialization affects the early averages. Adam corrects this with

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

For a constant gradient a , for example, $m_t = (1 - \beta_1^t)a$, so the correction recovers $\hat{m}_t = a$.

Normalize the direction average by coordinate scales

The update is

$$\theta_t = \theta_{t-1} - \eta \hat{m}_t \oslash (\sqrt{\hat{v}_t} + \epsilon).$$

Division $\oslash$, square roots, and addition of ϵ are elementwise. The term $\epsilon > 0$ prevents very small denominators. In diagonal-matrix notation,

$$D_t = \text{diag}\left(\frac{1}{\sqrt{\hat{v}_{t,i}} + \epsilon}\right), \quad \Delta\theta_t = -\eta D_t \hat{m}_t.$$

This is a changing diagonal preconditioner applied to momentum. The second moment measures squared-gradient scales; it is generally not the Hessian and does not directly measure distance to the minimum.[2]

These equations describe Adam's gradient update without weight decay. The result depends on history, so the size of a current gradient component alone does not determine the displacement.

14 / The effect of normalization

Adam's first step and coordinate scaling

Adam's first step can be calculated exactly. Since $m_0 = v_0 = 0$,

$$m_1 = (1 - \beta_1)a_1, \quad v_1 = (1 - \beta_2)(a_1 \odot a_1).$$

Bias correction gives $\hat{m}_1 = a_1$ and $\hat{v}_1 = a_1 \odot a_1$. The displacement in coordinate i is therefore

$$\Delta\theta_{1,i} = -\eta \frac{a_{1,i}}{|a_{1,i}| + \epsilon}.$$

If the component is nonzero and ϵ is negligible relative to its magnitude,

$$\Delta\theta_{1,i} \approx -\eta \operatorname{sign}(a_{1,i}),$$

where sign retains only the sign.

A small gradient need not give a tiny first step

For $a_1 = (100, 0.01)^\top$, ordinary gradient descent gives update magnitudes differing by 10^4 . Ignoring ϵ , Adam's first-step magnitudes are both η .

The smaller gradient is paired with a smaller denominator and therefore receives a comparable update, not a larger one. Later iterations also depend on historical directions, squared-gradient scales, and ϵ . This first-step example does not establish a general acceleration guarantee.

Normalization does not remove coordinate dependence

Return to $f(x, y) = \frac{1}{2}[(x-2)^2 + (y-1)^2]$. Its gradient at $(3, 2)$ is $(1, 1)^\top$. Let $x = cu$, $y = dv$, with $c, d > 0$. The gradient at the corresponding point $(3/c, 2/d)$ is $(c, d)^\top$.

Start Adam separately in both coordinate systems, with both moment states initialized to zero. Use learning rate η in the original coordinates and α in the new ones. Set $\epsilon = 0$ for this idealized first step; all gradient components in the example are nonzero.

The normalized original gradient is $(1, 1)^\top$, giving

$$\theta_1 = \begin{pmatrix} 3 - \eta \\ 2 - \eta \end{pmatrix}.$$

The normalized new gradient is also $(1, 1)^\top$, but mapping its displacement back still multiplies by c, d :

$$Az_1 = \begin{pmatrix} 3 - c\alpha \\ 2 - d\alpha \end{pmatrix}.$$

Agreement requires $c\alpha = \eta$ and $d\alpha = \eta$. When $c \neq d$, a common learning rate is still insufficient.

Normalizing a gradient and transforming a parameter displacement are different operations. Adam adjusts coordinate scales, but this does not give general coordinate invariance. Rotating the axes can also change its update.

15 / Rotation and Adam

Coordinatewise operations depend on the axes

Orthogonal rotation preserves Euclidean length, so ordinary gradient descent gives the same mapped update before and after rotation. Adam uses coordinatewise squaring, square roots, and division, which depend on the axes.

Use

$$Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}, \quad \theta = Qz.$$

At a point with gradient $a = (2, 1)^\top$, the rotated-coordinate gradient is

$$a' = Q^\top a = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 \\ -1 \end{pmatrix}.$$

Both original components are positive, while the second rotated component is negative. The objective has not changed: the same gradient has different components along different directions.

Calculate both first steps

Compare two Adam runs initialized with zero first and second moments, using the same learning rate η and $\epsilon = 0$ on these nonzero components. The direct first step is

$$\Delta\theta_{\text{direct}} = -\eta \text{sign}(a) = -\eta \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

The normalized vector in rotated coordinates is $(1, -1)^\top$. Mapping its displacement back gives

$$\Delta\theta_{\text{rotated}} = -\eta Q \text{sign}(Q^\top a) = -\eta Q \begin{pmatrix} 1 \\ -1 \end{pmatrix} = -\eta \begin{pmatrix} \sqrt{2} \\ 0 \end{pmatrix}.$$

The directions differ. Independently running Adam after merely rotating the axes can therefore produce a different trajectory.

For ordinary gradient descent, the same calculation gives

$$\Delta\theta = -\eta Q(Q^\top a) = -\eta a,$$

because $QQ^\top = I$. Adam differs because its coordinatewise normalization satisfies

$$Q \text{sign}(Q^\top a) \neq \text{sign}(a)$$

in this example.

The basis becomes part of the algorithm

A basis is a set of directions used to describe vectors. The columns of Q form an orthonormal basis, and $Q^\top a$ contains the coordinates of a in that basis.

If an algorithm treats each coordinate separately, its choice of coordinate directions matters. The quadratic example showed that some directions separate the curvatures more effectively. Applying coordinatewise scaling in those directions may express an update that a diagonal matrix in the original coordinates cannot.

This suggests selecting directions, scaling their components separately, and transforming the result back to the original parameters. The next section writes the complete operation as a matrix.

16 / Scaling in a new basis

Diagonal scaling can mix original directions

Fix an orthogonal matrix Q . Project the gradient a onto its column directions and scale the coordinates with a positive diagonal matrix D :

$$a' = Q^\top a, \quad u' = D a'.$$

Map the update back:

$$\Delta\theta = -\eta Q u' = -\eta \underbrace{Q D Q^\top}_P a.$$

Although D is diagonal, P generally is not. Treating components separately in a new basis can combine directions in the original coordinates.

Off-diagonal entries in two dimensions

With the preceding Q and $D = \text{diag}(d_1, d_2)$,

$$Q D Q^\top = \frac{1}{2} \begin{pmatrix} d_1 + d_2 & d_1 - d_2 \\ d_1 - d_2 & d_1 + d_2 \end{pmatrix}.$$

If $d_1 = d_2 = s$, the result is sI , so the basis has no effect. Unequal coefficients create off-diagonal entries: the first update component can depend on the second original gradient component.

For the quadratic in Section 11, choosing $D = \text{diag}(0.01, 1)$ gives

$$Q D Q^\top = \begin{pmatrix} 0.505 & -0.495 \\ -0.495 & 0.505 \end{pmatrix} = H^{-1}.$$

This is the ideal whitening correction. It used prior knowledge of the Hessian's eigenvectors and eigenvalues; Adam's historical second moments do not generally supply those exact coefficients.

Historical quantities also belong to a basis

Projection can act on a gradient or on momentum. For fixed Q , a linear moving average commutes with linear projection: averaging past gradients and then projecting gives the same first moment as projecting each gradient before averaging.

Elementwise squaring generally lacks this property:

$$(Q^\top a) \odot (Q^\top a) \neq Q^\top (a \odot a).$$

Historical squared-gradient statistics depend on their coordinate basis. If an optimizer changes that basis during training, it must specify how existing state is used in the new basis. Squared-gradient state cannot simply be rotated as though it were an ordinary gradient vector.

Choose a basis from computable statistics

SOAP estimates directions from Shampoo-style gradient statistics and applies Adam in the resulting basis.[3] Direction selection and coordinate-scale updates can then occur separately.

The shared idea with whitening is to select directions before scaling them. It does not require the full Hessian in advance, but the statistical directions need not be the Hessian's principal directions. For an actual model, both their usefulness and their maintenance cost matter.

17 / SOAP

Apply Adam along estimated directions

SOAP stands for Shampoo with Adam in the Preconditioner's eigenbasis. It estimates coordinate directions using Shampoo-style statistics, then applies Adam in those directions.^[3] For matrix parameters, the coordinate change acts on rows and columns.

Let a layer have weights $W \in \mathbb{R}^{m \times n}$ and gradient $G = \nabla_W \mathcal{L}$ of the same shape. Each G_{ij} is the partial derivative with respect to W_{ij} . Maintain

$$L \leftarrow \beta_2 L + (1 - \beta_2) G G^\top, \quad R \leftarrow \beta_2 R + (1 - \beta_2) G^\top G.$$

Here L is $m \times m$ and R is $n \times n$. For example, $(G G^\top)_{ij}$ is the inner product of gradient rows i, j , retaining row relationships not explicitly stored by elementwise squares.

Project, normalize, and map back

Estimate orthogonal eigenvector bases Q_L, Q_R from L, R . In the current bases,

$$G' = Q_L^\top G Q_R.$$

The left transform changes row directions; the right changes column directions. The result remains $m \times n$. With weights represented as $W' = Q_L^\top W Q_R$, this is the corresponding gradient.

Let M be the original-coordinate first moment and V the projected second moment. Omitting bias correction and weight decay, the main step is

$$\begin{aligned} M &\leftarrow \beta_1 M + (1 - \beta_1) G, & M' &= Q_L^\top M Q_R, \\ V &\leftarrow \beta_2 V + (1 - \beta_2) (G' \odot G'), \\ U' &= M' \oslash (\sqrt{V} + \epsilon), \\ W^+ &= W - \eta Q_L U' Q_R^\top. \end{aligned}$$

The direction U' is the Adam-style update in projected coordinates. The final multiplication maps its displacement back to parameter coordinates.^[3,4]

Directions and scales can update at different frequencies

Basis estimation is usually more expensive than elementwise updates. The bases can be refreshed periodically while V updates every step. A refresh must handle the coordinates of historical second moments. The first moment stays in original coordinates and is projected when used. The implementation also specifies initialization, numerical basis updates, bias correction, and AdamW-style weight decay.^[4]

For a fixed basis, projecting, scaling separately, and mapping back is the matrix-parameter version of the previous section's $Q D Q^\top$. The estimated directions need not match the true Hessian's principal directions, so this gives no exact whitening or universal optimality guarantee. Speed depends on the objective, gradient statistics, and the time and memory needed to maintain the bases.

18 / Coordinates, distance, and updates

How the methods fit together

The functions $f(x, y)$ and $g(u, v) = f(cu, dv)$ already described the same optimization problem at the start. Independently using ordinary gradient descent in each representation implicitly adopted each coordinate system's Euclidean length. Different units then produced different mapped updates.

Preserving a trajectory requires transforming the distance rule along with the coordinates. For the fixed invertible linear relation $\theta = Az$, the new metric is $A^\top A$, giving

$$z_{t+1} = z_t - \eta(A^\top A)^{-1} \nabla g(z_t).$$

Corresponding initial points and the same learning-rate sequence ensure $\theta_t = Az_t$. This is a concrete metric-gradient example and explains the geometric starting point of natural gradient.

Seeking faster optimization permits another distance rule and trajectory. In a narrow positive definite quadratic, the steeper direction restricts a shared learning rate, leaving slow progress in flatter directions. Rotation along principal directions followed by curvature-dependent scaling whitens the known quadratic. Ordinary descent then uses a common rate in whitened coordinates, equivalent to preconditioning by H^{-1} in the original ones.

During actual training, directions and scales must be estimated. Adam estimates coordinate scales from gradient history. Since it processes components separately, its update depends on the axes. SOAP additionally estimates a basis from gradient statistics, applies Adam there, and maps back to the original parameters.

The methods all involve choices of distance, direction, and scale, but they establish different properties. A fixed linear coordinate correction guarantees corresponding trajectories. Exact quadratic whitening illustrates ideal scale adjustment. Adam and SOAP construct adaptive updates from computable historical statistics. Their value must be judged by the actual cost of reaching a common target, rather than by the form of a coordinate transformation alone.

Further reading

These sources give the statistical background of natural gradient and the complete Adam and SOAP algorithms. The two-dimensional calculations in these notes provide concrete examples of how coordinates affect optimization.

[1] James Martens. *New Insights and Perspectives on the Natural Gradient Method*. Journal of Machine Learning Research, 21(146), 2020.

jmlr.org/papers/v21/17-678.html

[2] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. ICLR, 2015. Algorithm 1 gives moment estimates and bias correction.

arxiv.org/abs/1412.6980

[3] Nikhil Vyas et al. *SOAP: Improving and Stabilizing Shampoo using Adam*. arXiv:2409.11321v2, 2025. Section 4 and Algorithm 3 describe updates in projected coordinates.

arxiv.org/html/2409.11321v2

[4] Authors' SOAP implementation, including state and basis updates. Accessed September 15, 2026.

github.com/nikhilvyas/SOAP/blob/main/soap.py